

SAé 2.04

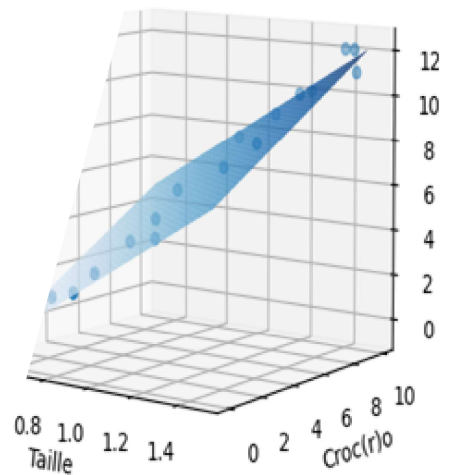
Partie 3 : Statistiques

Consignes, Outils

S2.04 - Exploitation d'une Base de Données

R2.08 - Statistiques

2025-2026



Plan du cours

Présentation de la partie Statistiques de la SAé

Livrable Partie 3 de la SAé : consignes générales

Consignes détaillées pour l'Oral

Exemples de données et de problématiques

L'Analyse de Données, c'est quoi ?

Boîte à Outils Python : Listes vs Numpy.Array

Boîte à Outils Python : les Pandas.DataFrame

• La SAé 2.04 : Exploitation d'une Base de Données

Données de la SAé 2.04

Travail sur les données de Parcoursup 2022 :

- Population : formations disponibles sur Parcoursup
- Informations sur la formation (lieu, capacités d'accueil, etc.)
- Informations sur les candidat.es et les admis.es dans chaque formation.

Étapes de la SAé 2.04

Le travail se fait en 3 parties successives, chacune avec un rendu à déposer sur Moodle :

- Partie 1 - Traduction et implantation d'un schéma relationnel
- Partie 2 - Peuplement et exploitation de la base de données
- **Partie 3 - Analyse statistique des données**

• Les Statistiques dans la SAé 2.04

Organisation des Statistiques pour la SAé 2.04

- **Une partie "théorique" de Statistiques sur la Régression Linéaire Multiple :**
Cours 3 et Fiche 3.
↪ outils qui seront à utiliser sur la Partie 3 (Statistique) de la SAé
- **Une partie "pratique" : travail sur la Partie 3 (Statistique) de la SAé.**
 - SAé-TPO : 2h-TP sur un extrait de données de la SAé (ParcoursupLannion.csv), qui servira de modèle pour la Partie 3 de la SAé.
 - 1h de projet encadré (par enseignant.e de BD) : construction d'une vue de la BD, choisie par chaque binôme (un extrait, du type de ParcoursupLannion.csv).
 - 1h de projet encadré (par enseignant.e de Statistiques) + 2h en autonomie : sur les données de votre vue, représentations graphiques, et régression linéaire multiple.

Évaluation de la Partie 3 (Statistique)

- **Contrôle de 20min sur papier**, pendant le dernier TD de la semaine du 15 au 19/06, portant sur la régression linéaire multiple (mise sous forme matricielle et calcul des paramètres).
- **Oral de 15min**, pendant les TD de la semaine du 22 au 26/06, présentant l'analyse statistique des données de votre vue.





Livable Partie 3 de la SAé : consignes générales

Le but de ce livrable est de réaliser une analyse du même type que celle faite dans le TPO sur les données de la vue ParcoursupLannion.csv.

• Conditions

Ce travail est à réaliser en binôme, **le même binôme que pour les précédents livrables de cette SAé.**

Vous réaliserez ce travail sur la **semaine du 15/06 au 21/06.**

Vous devrez déposer vos travaux sur Moodle avant le

Lundi 22/06 à 22h

• A rendre

Vous devrez déposer sur Moodle :

- Diaporama de présentation orale : voir les consignes ci-dessous.
- Le fichier .csv contenant votre Vue extraite de la Base de Données.
- Le fichier Python des commandes qui vous ont permis d'obtenir vos résultats.

• Etapes

1h encadrée par votre enseignant.e de BD + 1h encadrée par votre enseignant.e de Statistiques (semaine du 15/06)	Proposer une problématique (sérieuse ou absurde) pouvant être traitée à partir de données extraites de la Base de données de la SAé. Choix de la vue à concevoir. Elle devra comprendre au moins 5 variables , et un effectif total ≥ 20 . Réalisation de la Vue choisie
2h de SAé en autonomie (semaine du 15/06)	Représentations graphiques de vos données (votre vue), et régression linéaire multiple pour répondre à la problématique choisie. Réalisation d'un support de présentation orale. Voir consignes ci-après.
oral de 15min par binôme (semaine du 22/06)	Présentation (7-8min) + questions (7-8min). Voir consignes ci-après. Horaires communiqués ultérieurement.

• Critères d'évaluation

- Le suivi des consignes (chaque consigne est un point du barème).
- Pour la présentation orale : il y a des points attribués pour chaque partie et sous-partie détaillées ci-après. La partie indiquée comme + *difficile* comptera pour obtenir des points au-dessus de 16.
- La qualité/exactitude des raisonnements/interprétations statistiques.
- **La réponse aux questions lors de l'oral : chaque membre du binôme devra savoir expliquer chaque partie du code, et chaque raisonnement de statistiques.**

Si l'un.e des membres ne sait pas expliquer une partie du code / un raisonnement, il ne pourra pas avoir les points sur cette partie.

Il ne sera pas pris en compte dans l'évaluation le fait que les variables choisies soient ou non bien corrélées (vous ne pouvez pas le savoir à l'avance), mais seulement l'analyse des résultats.



Consignes détaillées pour l'Oral

• Consignes sur la présentation de vos résultats ($\approx$ 7-8min)

Voici le détail de ce qui devra apparaître dans votre présentation :

A. Les données - Problématique (1 ou 2 diapos, 1 ou 2min)

1. Présentation des données contenues dans votre vue (fichier csv) :
 - **la population** (effectif total d'au moins 20),
 - description des variables choisies (**au moins 4 variables**), chaque variable étant une colonne du .csv. Les variables doivent être **numériques**,
 - capture d'écran des premières lignes du .csv (avec noms des colonnes)
2. Enoncer une problématique en lien avec ces données, en précisant quelle variable vous souhaitez expliquer (la future variable endogène).

B. Représentations graphiques ($\approx$ 2 diapos, $\approx$ 2min)

Dans cette partie, vous devez illustrer les corrélations que vous voyez apparaître (ou non) dans la suite, en montrant :

- des boîtes à moustache (au moins 2 comparées),
- un/des nuages de points (au moins 1) à 2 variables.

Vous devrez commenter/interpréter les graphiques présentés (résumé écrit, un peu + oralement).

C. Régression linéaire multiple ($\approx$ 2 diapos, $\approx$ 2min)

1. Expliquer comment vous appliquez la régression linéaire multiple: choix de la variable endogène et des variables explicatives.
2. Expliquer en quoi la régression linéaire multiple avec ces choix de variables permettra de répondre à la problématique que vous avez énoncée dans la partie A.
3. Donner les paramètres obtenus pour votre régression linéaire multiple, et leur interprétation (influences positives/négatives).

D. Plus difficile ($\approx$ 1 diapo, $\approx$ 1min)

Calculer le coefficient de corrélation multiple de votre régression linéaire multiple. Interprétez-le.

E. Conclusion ($\approx$ 1 diapo, $\approx$ 1min)

1. Rappeler la problématique et proposer une réponse.
2. Justifier votre réponse à l'aide des paramètres/coefficient de corrélation obtenus.
3. Proposer des interprétations personnelles de vos résultats (sérieuses et/ou absurdes).

• La partie questions (≈ 7-8min)

Dans cette partie, vous montrerez à l'écran le code utilisé pour obtenir les résultats que vous avez présentés.

Chaque membre du binôme devra savoir expliquer chaque partie du code, et chaque raisonnement de statistiques.

Les questions pourront être du genre "que se passe-t-il si on change cette ligne dans le code ?" ou "que changer dans le code pour qu'il renvoie un résultat un peu différent ?".

Consignes sur la présentation du code

- Le code devra être lisible et compiler.
- Les parties du code correspondant aux différentes parties de la présentation devront être clairement séparés (partie A, partie B, partie C, etc.).
- Exceptionnellement, on demandera qu'il y aie peu de commentaires, pour laisser l'explication du code à l'oral.
- Dans votre code Python, les noms des variables désignant des np.array, des DataFrame ou des listes devront se terminer par un suffixe qui indique le type de la variable : ar pour les numpy.array, df pour les DataFrame, li pour les listes.



Exemples de données et de problématiques

• Exemple 1 (celui du TPO)

1. — Population : ensemble des formations post-bac de Lannion.
— Variable endogène : le pourcentage de filles dans les admis.es.
— Variables explicatives : le taux d'admission (nombre d'admis/nombre de candidats), la proportion de mentions B et celle de mentions TB au bac parmi les admis.es.
2. Problématique : le pourcentage de filles parmi les admis.es dans une formation lannionnaise peut-il être prédit en connaissant le taux d'admission de la formation, et la proportion de mentions B et TB au bac parmi les admis.es ?

• Exemple 2

1. — Population : ensemble des BUT Informatique de France.
— Variable endogène : le pourcentage de filles dans les admis.es.
— Variables explicatives : la latitude, la longitude, le nombre de "n" dans le nom de la commune.
2. Problématique : le pourcentage de filles admises en BUT Informatique peut-il être prédit en connaissant la latitude et la longitude de l'IUT, et le nombre "n" dans le nom de la commune de l'IUT ?

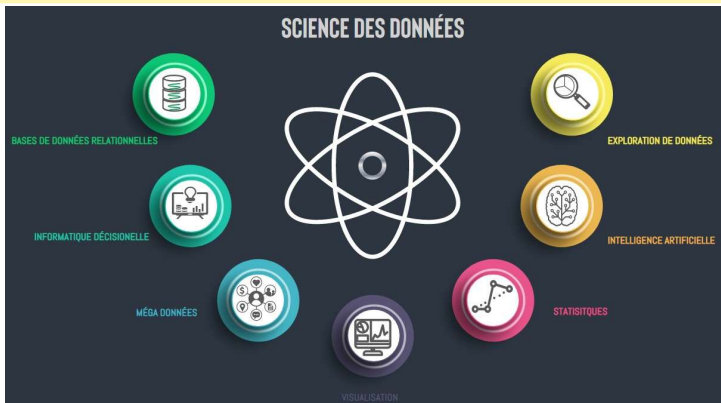


L'Analyse de Données, c'est quoi ?

Dans cette SAé : premiers pas en **Science des données**.

Définition

La **science des données** (*data science* en anglais) est un domaine interdisciplinaire qui utilise les mathématiques, les statistiques, le calcul scientifique, les méthodes scientifiques, les process, les algorithmes et les systèmes informatiques automatisés pour extraire et extrapoler des connaissances à partir de grandes quantités de données brutes.



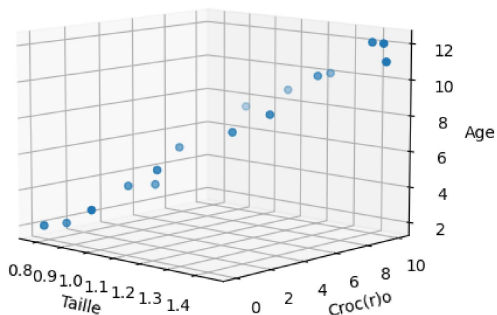
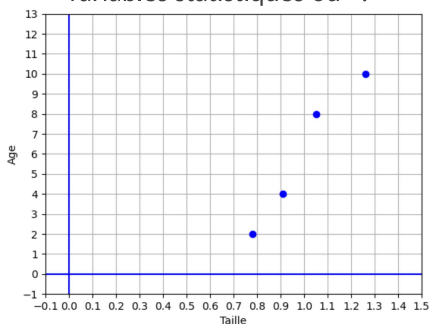
• Analyse de données

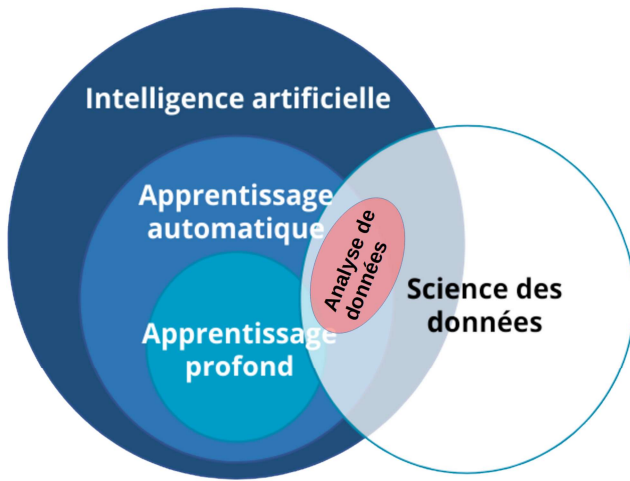
Dans la partie Statistique de la SAé, on sera plus particulièrement dans le domaine de **l'analyse de données**.

Définition

L'**analyse des données** est une famille de méthodes statistiques dont les principales caractéristiques sont d'être multidimensionnelles et descriptives.

"multidimensionnelles" : dans le Cours 3 on a vu des exemples de travail avec 3 variables statistiques ou +.





• Méthodes d'analyse de données

Il y a 3 principales catégories de méthodes d'analyse de données, qui se trouvent être les 3 catégories principales d'Apprentissage automatique (Machine Learning en anglais).

Ainsi, on retrouve ces 3 grandes catégories de méthodes par exemple dans la bibliothèque Python de Machine Learning Scikitlearn

scikit-learn

Machine Learning in Python

Getting Started

Release Highlights for 1.4

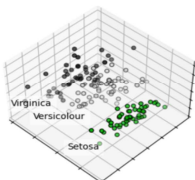
GitHub

- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Dimensionality reduction

Reducing the number of random variables to consider.

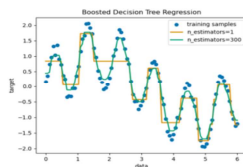
Applications: Visualization, Increased efficiency
Algorithms: PCA, feature selection, non-negative matrix factorization, and more...



Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.
Algorithms: Gradient boosting, nearest neighbors, random forest, ridge, and more...

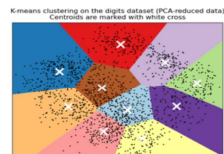


Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes

Algorithms: k-Means, HDBSCAN, hierarchical clustering, and more...



• Réduction de dimension

• Régression

• Clustering/Classification

Dans la partie Statistiques de la SAé, on va utiliser la Régression Linéaire Multiple (vue dans le Cours 3), qui est la + courante des méthodes de régression.



Numpy.Array : une structure supplémentaire sur certaines listes

Structure qui s'applique sur

- des listes d'objets de même type
- ou des listes de listes de même longueur

```
In [2]: liste1=[2,3,5]
```

```
In [3]: liste1  
Out[3]: [2, 3, 5]
```

```
In [4]: array1=np.array(liste1)
```

```
In [5]: array1  
Out[5]: array([2, 3, 5])
```

```
In [9]: liste3=[[2,1],[3,5]]
```

```
In [10]: liste3  
Out[10]: [[2, 1], [3, 5]]
```

```
In [11]: np.array(liste3)  
Out[11]:  
array([[2, 1],  
       [3, 5]])
```

↪ ne s'applique pas sur une liste de listes de longueurs différentes :

```
In [6]: liste2=[[2],[3,5]]
```

```
In [7]: liste2  
Out[7]: [[2], [3, 5]]
```

```
In [8]: array2=np.array(liste2)  
C:\Users\tiphaine\AppData\Local\Temp\ipykernel_  
VisibleDeprecationWarning: Creating an ndarray
```

Retour simple au type liste

```
In [17]: array1.tolist()  
Out[17]: [2, 3, 5]
```

```
In [18]: array3.tolist()  
Out[18]: [[2, 1], [3, 5]]
```

Points communs : slicing, mutabilité

Accéder à un élément

- liste ou np.array à 1 dimension :
- liste de listes ou np.array à 2 dimensions :

```
In [22]: liste1[0]  
Out[22]: 2
```

```
In [23]: array1[0]  
Out[23]: 2
```

```
In [24]: liste3[0][1]  
Out[24]: 1
```

```
In [25]: array3[0][1]  
Out[25]: 1
```

```
In [26]: array3[0,1]  
Out[26]: 1
```

Modifier un élément

- liste ou np.array à 1 dimension :
- liste de listes ou np.array à 2 dimensions :

```
In [27]: liste1  
Out[27]: [2, 3, 5]
```

```
In [28]: liste1[0]=10
```

```
In [29]: liste1  
Out[29]: [10, 3, 5]
```

```
In [31]: array3[0,1]=10
```

```
In [32]: array3  
Out[32]:  
array([[ 2, 10],  
       [ 3, 5]])
```

Différences : les opérations + et * sur les listes et les np.array

Opérations +, * sur les listes

Pour les listes, + et * se réfèrent à la concaténation :

```
In [34]: liste1
Out[34]: [10, 3, 5]

In [35]: liste1+liste1
Out[35]: [10, 3, 5, 10, 3, 5]
```

```
In [36]: liste1*2
Out[36]: [10, 3, 5, 10, 3, 5]

In [37]: liste1*3
Out[37]: [10, 3, 5, 10, 3, 5, 10, 3, 5]
```

Opérations +, * sur les np.array

Pour les np.array, + et * sont les opérations numériques sur les éléments :

```
In [38]: array1
Out[38]: array([2, 3, 5])

In [39]: array1+array1
Out[39]: array([ 4,  6, 10])
```

```
In [40]: array1*2
Out[40]: array([ 4,  6, 10])

In [41]: array1*3
Out[41]: array([ 6,  9, 15])
```

Quelques conséquences de ces différences

Création d'une liste / d'un numpy.array

```
In [54]: liste4=[]
...: for i in range(4):
...:     liste4 += [i]

In [55]: liste4
Out[55]: [0, 1, 2, 3]
```

```
In [56]: array4=np.zeros(4)

In [57]: array4
Out[57]: array([0., 0., 0., 0.])

In [58]: for i in range(4):
...:     array4[i] = i

In [59]: array4
Out[59]: array([0., 1., 2., 3.])
```

- Les opérations numériques, et donc les fonctions de statistiques, s'utilisent de manière + sûre sur des np.array.
→ On verra la semaine prochaine les fonctions numpy pour la moyenne, la variance etc. , qui s'appliquent à des np.array.
- La concaténation de np.array est + compliquée, il faut utiliser la fonction np.concatenate.



Boite à Outils Python : les Pandas.DataFrame

C'est une structure qui se rajoute sur un `np.array` pour créer un tableau avec des noms de lignes et de colonnes :

	Maths	Prog	Com
Albert	1	2	3
Maria	12	13	10
Zoe	15	12	13

De même que les array sont dans la librairie Numpy, la structure `dataframe` se trouve dans une librairie Python : la librairie Pandas.

```
import pandas as pd
```

Créer un DataFrame à partir d'un fichier .csv

Pour un fichier `MonFichier.csv` se trouvant dans le même dossier que le fichier Python utilisé :

```
MonDataFrame=pd.read_csv("MonFichier.csv")
```

De np.array à DataFrame et inversement

Pour créer un DataFrame à partir d'un `np.array`, il faut créer la liste des noms des lignes, et la liste des noms des colonnes, puis les combiner ainsi :

```
In [71]: array5
Out[71]:
array([[ 1,  2,  3],
       [12, 13, 10],
       [15, 12, 13]])

In [72]: noms_lignes=['Albert','Maria','Zoe']

In [73]: noms_colonnes=['Maths','Prog','Com']

In [74]: df5=pd.DataFrame(data=array5,index=noms_lignes,columns=noms_colonnes)

In [75]: df5
Out[75]:
   Maths  Prog  Com
Albert    1    2    3
Maria   12   13   10
Zoe     15   12   13
```

Pour revenir à la structure `np.array` à partir d'un DataFrame, c'est simple :

```
In [76]: array5=df5.to_numpy()

In [77]: array5
Out[77]:
array([[ 1,  2,  3],
       [12, 13, 10],
       [15, 12, 13]])
```